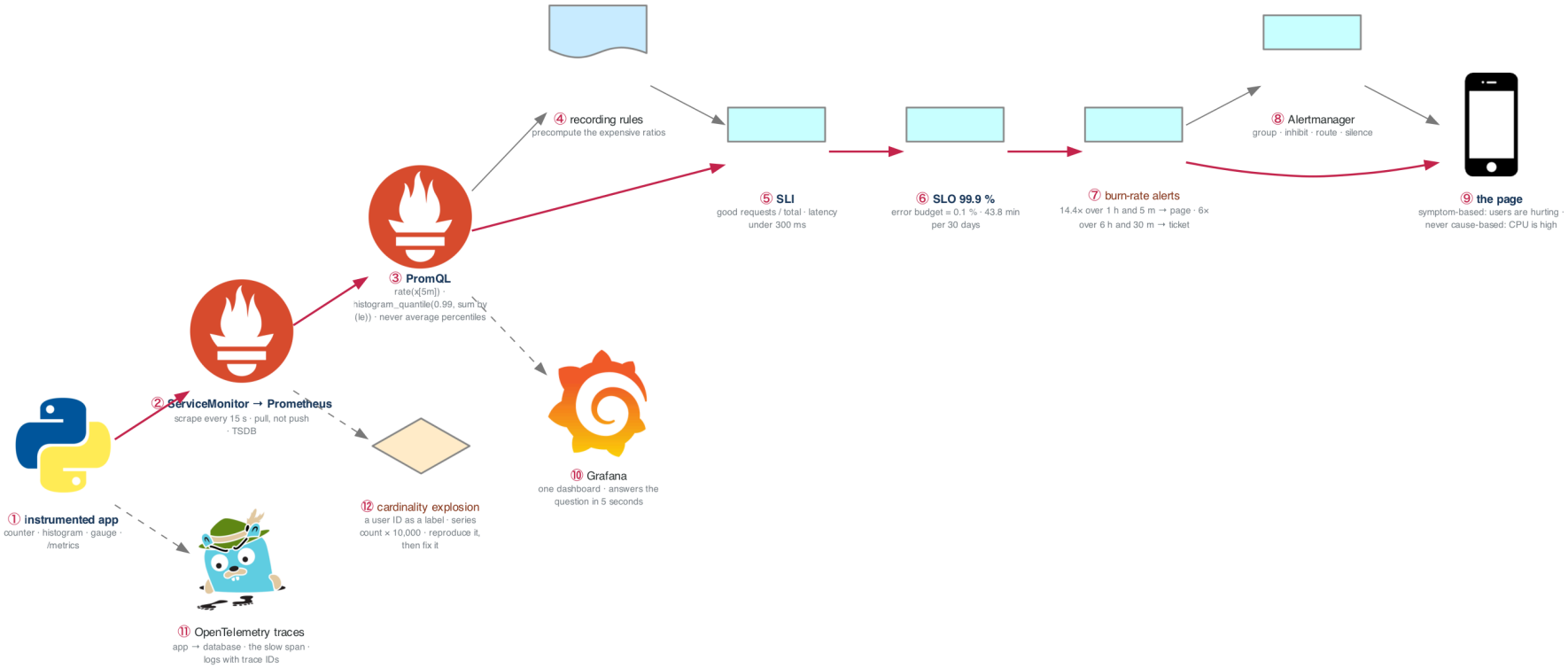
From a request to an alert that means users are hurting, numbered

Week 20 · Figure 20.1 · figure with badges 1–12 · 2026-10-17



What to say, badge by badge

About four minutes at a normal pace (530 words). Point at each badge as you say its line.

Open
“A dashboard passes one test: can someone tell users are having a bad time within five seconds of looking at it. This figure is the path from a line of instrumentation to a page that fires only when that is true. Follow the blue line; the numbers in badges 6 and 7 are the ones to memorize.”

1 instrumented app
“Three metric types cover almost everything. A counter for requests and errors, a histogram for latency with buckets you chose deliberately, a gauge for things that go up and down. Exposed at /metrics in the format you previewed in week 4.”

2 ServiceMonitor and Prometheus
“Prometheus pulls. A ServiceMonitor tells it which Services to scrape, every 15 seconds by default. Samples land in a time-series database keyed by metric name and labels. Pull means the app does not need to know where Prometheus is.”

3 PromQL
“`rate` over a window turns a counter into a per-second rate. For percentiles, sum the histogram buckets keeping `le`, then `histogram_quantile`. Averaging 99th percentiles across instances is wrong and the most common mistake in the drill. Ten queries from scratch, no copy-paste.”

4 recording rules
“Precompute the ratios that dashboards and alerts both need. One rule evaluated every 30 seconds beats fifty dashboards computing the same thing.”

5 SLI
“A service level indicator is a ratio: good events over total. Requests under 300 milliseconds over all requests. Pick the one that tracks user experience, not the one that is easy to measure.”

6 SLO
“The target: 99.9 percent of requests good, over 30 days. The error budget is the other 0.1 percent, which is 43.8 minutes a month. Compute it by hand once. The budget is what makes risk a number you can spend.”

7 burn-rate alerts
“Burn rate is how fast you are spending the budget. A rate of 14.4 over the last hour, confirmed by the last 5 minutes, means the whole month's budget goes in two days: page. A rate of 6 over 6 hours, confirmed by 30 minutes: open a ticket. Two windows each, so a blip does not page and a slow burn is not missed.”

8 Alertmanager
“Groups related alerts into one notification, inhibits the noisy ones when a parent fires, routes by severity, and lets you silence during maintenance. Alert design is here as much as in PromQL.”

9 the page
“Symptom-based: the error rate or latency users see. Never cause-based: CPU high, disk at 80 percent. Causes go on the dashboard for the person who was paged, not on the pager.”

10 Grafana
“One dashboard, top row answers the question. Error rate, latency percentiles, traffic, saturation. If the five-second test fails, remove panels until it passes.”

11 OpenTelemetry traces
“A trace follows one request across services. The span that is slow is the one you fix. Logs carry the trace ID so you can jump from a log line to its trace.”

12 cardinality explosion
“Put a user ID in a label and every user becomes a new series. Ten thousand users, ten thousand times the memory. Reproduce it, watch Prometheus suffer, then fix it by moving the ID to the trace or the log where it belongs.”

Close
“Instrument, scrape, query, define good, set a budget, alert on the burn. The page means users are hurting; everything else is on the dashboard for the person who answers it.”